OpenZeppelin | security

OpenZeppelin Compact Contracts Audit



October 19, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 6

Security Model and Trust Assumptions 8

Additional Considerations 9

Medium Severity 10

M-01 Sending to the Treasury Itself Strands the Sent Coin 10

Low Severity 11

L-01 Proposal.state Conflates Lifecycle Stage and Expiry 11

L-02 Depositors Can Unmask the Hidden Forwarder Parent and Trace Treasury Spends 11

L-03 Pending Treasury Sends Can Be Invalidated by Any Depositor 12

L-04 NativeShieldedToken Contract-Held Tokens Can Only Be Burned 13

Notes & Additional Information 14

N-01 Example Contract Does Not Expose Token Metadata Circuits 14

N-02 Inconsistent Placement of Domain Tags in Hash Preimages 14

N-03 Inconsistent Initialization Checks in ConfidentialFungibleToken 15

N-04 Unchecked Spender Key Lookup in _spendEscrow 15

N-05 Unused Proposal State in the Multisig Preset 15

Conclusion 17

Appendix 18

Issue Classification 18

Summary

Type	Privacy
Timeline	From 2026-09-30 To 2026-10-15
Languages	Compact

Total Issues	10 (0 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	1 (0 resolved)
Low Severity Issues	4 (0 resolved)
Notes & Additional Information	5 (0 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited the OpenZeppelin/compact-contracts repository across three scopes.

Multisig modules: the multisig modules and the cryptographic and encoding utilities they depend on were audited at commit f270f73 (release `v0.4.0-alpha.6`, Ledger v9). The following files were in scope:

```
contracts
└── src
    ├── crypto
    │   ├── Ecdsa.compact
    │   └── Eip712.compact
    ├── multisig
    │   ├── examples
    │   │   ├── ForwarderPrivateExample.compact
    │   │   ├── ForwarderShieldedExample.compact
    │   │   ├── ForwarderUnshieldedExample.compact
    │   │   ├── NativeShieldedTokenIssuerExample.compact
    │   │   └── ShieldedMultiSigV2Example.compact
    │   ├── presets
    │   │   ├── NativeShieldedTokenIssuer.compact
    │   │   └── ShieldedMultiSigV2.compact
    │   ├── ForwarderPrivate.compact
    │   ├── ForwarderShielded.compact
    │   ├── ForwarderUnshielded.compact
    │   ├── ProposalManager.compact
    │   ├── Secp256k1EcdsaMultisig.compact
    │   ├── ShieldedTreasury.compact
    │   ├── ShieldedTreasuryStateless.compact
    │   ├── Signer.compact
    │   └── UnshieldedTreasury.compact
    └── utils
        └── EvmAbi.compact
```

Owner-controlled token preset: the `ZOwnableNativeShieldedToken` preset and its example contract were audited in two versions, one per Midnight ledger: commit 5cc68d3 (release `v0.3.0-rc.2`, Ledger v8) and commit f270f73 (release `v0.4.0-alpha.6`, Ledger v9). The following files were in scope in both versions:

```
contracts
└── src
    └── token
        ├── examples
        │   └── ZOwnableNativeShieldedTokenExample.compact
```

```
        └── presets
            └── ZOwnableNativeShieldedToken.compact
```

Fixes for the previous audit: the fixes for the High and Medium severity issues of the previous audit of the library (H-01 to H-03 and M-01 to M-04), together with the fix for L-01 on which the H-02 fix depends, were reviewed as a diff from the previously audited commit 0e9d659 to commit 915a487 (release `v0.3.0-rc.1`). The changes made to the following files were in scope:

```
contracts
└── src
    ├── crypto
    │   └── ElGamal.compact
    ├── token
    │   ├── extensions
    │   │   ├── NativeShieldedTokenFamilyPublicSupply.compact
    │   │   └── NativeShieldedTokenPublicSupply.compact
    │   ├── ConfidentialFungibleToken.compact
    │   ├── NativeShieldedToken.compact
    │   ├── NativeShieldedTokenCore.compact
    │   ├── NativeShieldedTokenFamily.compact
    │   └── NonFungibleToken.compact
    └── utils
        └── Utils.compact
```

In addition, the fixes for N-07 and N-08 of the previous audit, which modify the `ZOwnablePK` module composed by the token preset, were reviewed in both preset commits listed above (#1055 and #1057 for Ledger v8, #1054 and #1056 for Ledger v9).

System Overview

The Compact Contracts library provides reusable modules for building contracts on Midnight, a network whose contracts execute as zero-knowledge circuits written in the Compact language. As in the previous audit, modules are not deployable on their own. A contract composes them by importing each under a prefix and delegating to it explicitly, decides which circuits become entry points, and supplies all access control.

This engagement covers a new family of modules for ECDSA-based, threshold-controlled multisig custody and token issuance. It is built on new cryptographic primitives for in-circuit secp256k1 ECDSA verification, EIP-712 typed-data hashing and EVM ABI encoding, together with a proposal manager that tracks the lifecycle and expiry of pending transfers. It also covers forwarders that route shielded, unshielded and privately committed funds to a designated parent, and treasuries that hold shielded and unshielded coins. On the token side, the scope includes a native shielded token preset controlled by a single hidden owner, and the supply-tracking and derived-nonce extensions for native shielded tokens. Finally, the engagement reviews the fixes made to the library since the previous audit.

Threshold signing: `Signer` keeps a registry of signer identities and a threshold, and `Secp256k1EcdsaMultisig` builds a `t`-of-`n` multisignature over secp256k1 ECDSA on top of it. Each signer is registered as a commitment to its public key and a per-instance salt, so the keys themselves never appear on-chain, although every approved call discloses which commitments approved it. The threshold `t` is fixed when the module is imported, and every gated call carries exactly `t` public keys and signatures, which are checked to be pairwise distinct, registered, and valid. Signatures must use the low-`s` encoding enforced by the `Ecdsa` module, so each signature has a single valid form. The signer set is fixed at construction: no circuit adds, removes, or rotates a signer, or changes the threshold. The message that signers approve is built by the consuming contract. `Eip712` and `EvmAbi` reproduce the EIP-712 typed-data digest that Ethereum signing infrastructure produces, so existing EVM signers can authorize Midnight operations. Since no network identifier is available in-circuit and a Midnight address does not fit the EIP-712 `address` type, binding a signature to one deployment is achieved by placing the contract address inside the signed struct.

Multisig presets: `ShieldedMultiSigV2` is a shielded treasury controlled by the threshold signers. Any party can deposit a shielded coin, and a single `execute` transaction carrying `t` approvals sends value from a coin to a coin public key. The approved digest covers the

contract address, a replay counter, the recipient, the token color, and the amount. The preset composes `ShieldedTreasuryStateless`, which stores no coins on the ledger, so an off-chain operator supplies the coin to spend as a circuit argument. The plaintext of a deposited coin (its nonce, color, and value) cannot be recovered from chain data, so the operator must obtain it from the depositor or through a deposit channel that it controls. The indexer is then only used to locate the coin's position in the commitment tree.
`NativeShieldedTokenIssuer` is a token issuer under the same signer model, intended for tokenized deposits. Value enters only through threshold-approved mints, either to a holder's coin public key or to a coin held by the contract, and leaves only through threshold-approved burns.

Treasury, proposal, and forwarder modules: these modules are offered as building blocks rather than composed by a preset. `ShieldedTreasury` stores one merged coin per color on the public ledger, so the contract can spend without the caller supplying a coin, at the cost of publishing its holdings. `UnshieldedTreasury` tracks balances of unshielded tokens. `ProposalManager` records proposals for multi-step approval flows, and every proposal (recipient, color, amount, and state) is public. The forwarders receive deposits for a fixed parent recipient. `ForwarderShielded` and `ForwarderUnshielded` forward each deposit to a public parent within the same transaction, whereas `ForwarderPrivate` accumulates deposits under a hashed commitment to a parent coin public key and releases them to an operator presenting the commitment's preimage.

Owner-controlled token preset: `ZOwnableNativeShieldedToken` composes `NativeShieldedToken`, `ZOwnablePK`, and the public supply extension into a native shielded token with a single owner who mints and burns. The owner is stored as a commitment, and ownership is proven by knowledge of a coin public key and a secret nonce that reproduce it. Holders transfer coins directly through Zswap, without any contract call. Mint amounts and the cumulative minted total are public, while mint recipients and burns are not. The preset is published for both Ledger v8 and Ledger v9.

Fixes to the previous audit: the reviewed fixes add domain separation to the scalar derived from an account secret, make memo clearing conditional on an epoch counter that the recipient cannot reset, require the encryption secret when sweeping and clearing memos, widen the balance claimed on a spend, derive the escrow owner memo from a per-pair counter, restrict native shielded token mints and burns to coin public key recipients, and require token existence when approving in `NonFungibleToken`. The fixes for N-07 and N-08 reject a zero instance salt in `ZOwnablePK` and document that the secret nonce is the sole credential of its owner.

Security Model and Trust Assumptions

Composing contract: as in the rest of the library, the composing contract remains the primary trust boundary. Every underscore-prefixed circuit in the multisig modules, including treasury sends, signer registration, and proposal transitions, carries no access control and must be gated by the consumer. `Signer` has no initializer, so a consumer that leaves signer registration ungated lets the first caller install their own signer set. Recipients are not validated by the treasury modules, and integrators are expected to prevent sends to recipients that cannot spend the funds.

Signers: the registered signers collectively authorize every value movement in the multisig presets, and no on-chain mechanism limits, delays, or reverses an approved operation. Because the signer set and threshold cannot be changed, losing more than `n - t` keys permanently freezes every gated circuit, and compromising `t` keys gives full control with no on-chain recovery short of a circuit upgrade. Signers are assumed to verify each EIP-712 payload before signing, since the digest is the only description of the operation that they approve.

Operator: the party submitting `execute`, and the operator of a `ForwarderPrivate` deployment, are not specific on-chain roles. In `ShieldedMultiSigV2`, any party holding valid approvals can submit them, and the operator is assumed to obtain and retain the plaintext of every coin that the contract receives. Losing that data leaves the corresponding funds unspendable, so the preset is intended for deployments where the operator controls the channel through which deposits are made, rather than for open public deposits. Since every `execute` reads and increments the same replay counter, at most one execution can succeed per block, and a transaction that loses this race must be re-proven. In `ForwarderPrivate`, knowledge of the operational secret is the only authorization for draining deposits, so its holder is trusted with every deposit made to the forwarder.

Token owner: the owner of a `ZOwnableNativeShieldedToken` controls minting and burning outright. The secret nonce is the only credential: `ownPublicKey` is supplied by the prover and does not bind the submitting wallet, so whoever holds the nonce and the owner's coin public key can act as the owner. Ownership transfer completes in a single step, so an incorrect new owner identifier loses control of the token permanently, and renouncing ownership disables minting and burning permanently.

Contract Maintenance Authority: a deployed contract's verifier keys can be replaced by its maintenance authority, which therefore sits above the signers of a multisig preset and the owner of the token preset. Any property established in this report holds for the circuits deployed at the time of review and for as long as that authority is trusted.

Wallet, witness, and proof server: these off-chain components are assumed to supply fresh randomness for every mint nonce and instance salt, to compute signer commitments and owner identifiers with the exported pure circuits rather than reimplementations, to retain the coin information returned by mints, sends, and burns, and to encrypt shielded outputs to the intended recipient's encryption key. The proof server sees every secret witness, including the owner's secret nonce, and is assumed to be run where the secret holder controls it.

Midnight platform: the compiler, runtime, and ledger are treated as an external dependency. Signature verification relies on the platform's `secp256k1EcdsaVerify` and `keccak256` primitives, which require the ZKIR v3 backend and were not reviewed. The standard library is relied upon for coin handling, including the automatic claiming of a coin that a contract sends to itself. Transaction atomicity is assumed, so a failed assertion leaves no partial state.

Additional Considerations

- Ledger v9, on which the reviewed multisig modules and one version of the token preset depend, was not production-ready at the time of the audit. The ECDSA-based multisig relies on `keccak256` and secp256k1 verification, which require the ZKIR v3 backend. The Ledger v8 release contains an earlier version of the multisig modules that was not part of this audit.
- On the Ledger v9 node used for testing, transactions that receive and send unshielded tokens in a single call, including `ForwarderUnshielded` deposits, are currently rejected by the ledger's dismiss-time limit. A fix is planned on the ledger side.
- Some issues identified in `v0.4.0-alpha.5` were already fixed by `v0.4.0-alpha.6` and are therefore not included in this report.

Medium Severity

M-01 Sending to the Treasury Itself Strands the Sent Coin

The `_send` circuit of `ShieldedTreasury` spends the stored coin of the requested color, sends `amount` to `recipient`, and records any change in `_coins`. The recipient is not validated, so the address of the contract itself is accepted. In that case, the standard library `sendShielded` circuit claims the receipt of the sent coin on behalf of the contract, so the contract owns that coin at the protocol level. However, `_send` records only the change and increments `_sent` by the full amount, as though the value had left the treasury.

Since `_coins` holds one coin per color and the module only spends the coins recorded there, no `ShieldedTreasury` circuit can spend the sent coin afterwards. The coin cannot be recorded later either, because `insertCoin` requires the Merkle tree index of the coin to have been allocated within the current transaction, and the receipt was claimed in an earlier one. As a result, `getTokenBalance` and `getReceivedMinusSent` both under-report the holdings by `amount`, and the funds can only be recovered by a consuming contract that defines its own circuit to spend the coin directly or to write it into the exported `_coins` map. The equivalent hazard in `UnshieldedTreasury` is documented, but `ShieldedTreasury` does not mention it.

Since `ShieldedTreasury` is a library module, the authorization policy and the source of `recipient` are defined by each consuming contract, and any authorized send that names the contract address removes the funds from the treasury permanently unless that contract anticipated the case. In contrast, `ShieldedTreasuryStateless` is not affected, because the operator supplies the coin at spend time and a self-sent coin therefore remains spendable.

Consider rejecting a recipient equal to the address of the contract in `_send`, since a self-addressed send cannot move value. Alternatively, consider merging the sent coin back into `_coins` within the same transaction and excluding it from `_sent`, or at a minimum documenting the hazard as `UnshieldedTreasury` does.

Low Severity

L-01 `Proposal.state` Conflates Lifecycle Stage and Expiry

In `ProposalManager`, the `state` field of `Proposal` stores two unrelated values: the proposal's expiry timestamp while it is pending, and the executed or cancelled sentinel (`1` or `2`) once it reaches a terminal stage. The `Proposal` documentation describes this encoding, but reaching a terminal stage overwrites the expiry, so the deadline of every executed or cancelled proposal is lost, and every off-chain reader of `_proposals` must apply the sentinel rules to tell a stage from a timestamp. This overlay is also the only reason `_createProposal` asserts `expiry > cancelledState()`, a comparison between a time in seconds and a lifecycle marker. That guard depends on `cancelledState()` remaining the largest sentinel, so adding a new stage would require it to be revisited. Such semantic overloading is error-prone for a library that other projects build on.

Consider storing the lifecycle stage as a `ProposalStatus` value in a field separate from the expiry.

L-02 Depositors Can Unmask the Hidden Forwarder Parent and Trace Treasury Spends

`ForwarderPrivate` promises that its parent "stays hidden on-chain", and `ShieldedMultiSigV2` promises "full balance privacy". Both rely on coin commitments being opaque to anyone who does not know the coin's nonce.

However, both contracts spend through the standard library `sendShielded` circuit, which derives the output nonces from the input coin's nonce with a public hash. This is a property of the standard library rather than of these modules, so every contract that spends through `sendShielded` is exposed in the same way. Whoever knows a spent coin therefore knows the nonces of its outputs, and every depositor knows the coin they deposited. A depositor can:

1. Find the spending transaction, since a contract-owned coin's nullifier depends only on the coin and the public contract address.
2. Recover the amount sent. It equals the coin value on a full spend, and on a partial spend it can be brute-forced from the change commitment, whose value is its only unknown, when amounts fall in a searchable range.

3. Test a set of candidate coin public keys against the sent-coin commitment. For `ForwarderPrivate`, this reveals the parent if it is among the candidates.
4. Repeat with the now fully known change coin, tracing each later `ShieldedMultiSigV2` spend that descends from it and the balance it carries.

Consider raising with the Compact team a `sendShielded` variant that derives output nonces from caller-supplied secret material, which would prevent steps 2 to 4 for every consumer. Alternatively, these spends could be built from `createZswapInput`, `createZswapOutput` and the matching kernel claims, deriving output nonces from secret material such as a hash of `opSecret` and the input nonce for `ForwarderPrivate`, or a secret witness for `ShieldedMultiSigV2`, although this requires reimplementing the coin commitment and nullifier derivations that the standard library does not export. Step 1 is inherent to contract-owned coins. It is already noted for `ShieldedTreasuryStateless` and should be documented for `ForwarderPrivate` and `ShieldedMultiSigV2` as well. At a minimum, consider documenting this limitation in both modules.

L-03 Pending Treasury Sends Can Be Invalidated by Any Depositor

Midnight transactions are proven against the ledger state that exists at proving time. A ledger read inside a transaction is checked again when the node applies it, and the whole call is rejected if the value has changed. `ShieldedTreasury._send` reads and spends the single stored coin of a color. However, every `_deposit` of the same color merges into that coin and replaces it. The merge happens even for a zero-value deposit. If the ledger accepts zero-value shielded outputs, such a coin of any color needs no input and costs only the transaction fee. Otherwise, a deposit of one unit of the token has the same effect. Likewise, `UnshieldedTreasury._send` reads `_balances[color]`, which any non-zero deposit changes. A zero-value deposit rewrites the same value and therefore does not invalidate a pending send.

A consumer that exposes a permissionless deposit therefore allows anyone to invalidate every authorized send of a color that is waiting to land, block after block. Ordinary deposits have the same effect, so this situation may arise incidentally. Authorized sends can be slow to prove, such as when they include signature verification, so the attacker can keep payouts from executing for as long as they keep paying fees. When combined with `ProposalManager`, a delayed proposal can also expire and require a new approval round. No funds are lost. The `_deposit` documentation presents zero-value deposits as a harmless signaling mechanism

and does not mention this effect. `ShieldedMultiSigV2` is not affected, since it composes `ShieldedTreasuryStateless`, whose deposits write no state that `execute` reads.

Consider restructuring the modules so that deposits never write the state that a pending send reads. For example, `ShieldedTreasury` could store each deposit as a separate entry and let `_send` spend a chosen entry, and `UnshieldedTreasury` could record credits in a structure that `_send` does not read back exactly. At a minimum, consider documenting that a permissionless `_deposit` makes `_send` freezable, and rejecting zero-value shielded deposits.

L-04 `NativeShieldedToken` Contract-Held Tokens Can Only Be Burned

The H-03 fix added `_mintToSelf` to `NativeShieldedToken`, which lets the token contract hold its own coins. However, the only circuit that spends a contract-held coin is `_burnFromSelf`. The `NativeShieldedTokenIssuer` preset exposes exactly this pair. Its signers can therefore mint a reserve that can never be sent anywhere, only destroyed.

Moving such coins requires composing a custody module such as `ShieldedTreasury`, which neither the token modules nor the preset mention. Furthermore, that composition does not work today. The standard library already claims a self-minted coin, and `ShieldedTreasury._deposit` claims it a second time. The resulting duplicate output makes the node reject the transaction. The only workaround is a direct `insertCoin`, which skips the `_received` accounting. This gap is tracked upstream in compact-contracts#1112.

Consider removing the ability to hold tokens from the token modules. Alternatively, consider documenting in `NativeShieldedToken` and `NativeShieldedTokenIssuer` that contract-held coins are only usable once a custody module is composed that records them without claiming them a second time through `receiveShielded`.

Notes & Additional Information

N-01 Example Contract Does Not Expose Token Metadata Circuits

The `ZOwnableNativeShieldedTokenExample` contract re-exports every circuit of the `ZOwnableNativeShieldedToken` preset except the `name`, `symbol`, and `decimals` circuits, even though they are intended to be user-facing. The underlying `_name`, `_symbol`, and `_decimals` ledger fields remain exported and readable off-chain, but the deployed contract offers no circuit through which the token metadata can be queried, unlike the comparable `totalMinted` and `tokenColor` values. The same omission is present in the Ledger v8 version of the example.

Consider exporting the `name`, `symbol`, and `decimals` circuits from the example contract for completeness.

N-02 Inconsistent Placement of Domain Tags in Hash Preimages

The in-scope modules separate their `persistentHash` domains with a fixed tag, but they place that tag in different positions. `_calculateParentCommitment` in `ForwarderPrivate` puts `"ForwarderPrivate:commitment"` first, whereas `calculateSignerId` in `Secp256k1EcdsaMultisig` and `_computeOwnerCommitment` in `ZOwnablePK` put `"Secp256k1EcdsaMultisig:signer"` and `"ZOwnablePK:shield:"` last. No collision is possible today, since the preimages either differ in length or, as with the two four-word preimages, carry different tags in the same position. However, a mixed convention makes it harder to see at a glance that preimages from different domains can never match, especially as more hashed structures are added.

Consider adopting a single convention, such as placing the domain tag first, for new hashed structures. Since moving the tag in an existing derivation changes every value derived from it, any such change to `Secp256k1EcdsaMultisig` or `ZOwnablePK` should be treated as a breaking change.

N-03 Inconsistent Initialization Checks in `ConfidentialFungibleToken`

`ConfidentialFungibleToken` intends every exported circuit to call `assertInitialized`, either directly or through a shared internal helper such as `_spendEscrow` or `_debit`. However, this policy is not documented, and two exported views do not follow it. `allowance` and `isRegistered` skip the check, whereas the equivalent `balanceOf` and `pendingOf` views perform it. Likewise, `transferFrom` checks initialization directly and then again inside `_spendEscrow`, as `transfer` and `_move` also do through `_debit` and `_credit`, while `_refundPriorEscrow` does not check at all and relies on its only caller, `approve`, having done so. No state can be corrupted, as the unchecked exported circuits only read, but the mixed pattern forces each reviewer to re-derive whether a given circuit is covered.

Consider adding `assertInitialized` to `allowance` and `isRegistered`, and documenting the policy in the module header.

N-04 Unchecked Spender Key Lookup in `_spendEscrow`

The `_spendEscrow` circuit of `ConfidentialFungibleToken` reads the spender's encryption key from `_encryptionKeys` without first asserting that the entry exists. This is safe today: `approve` only creates an escrow for a registered spender, and keys are never removed. However, the guarantee lives in a different circuit. A future change, such as key removal or another path that creates escrows, could leave `_spendEscrow` reading a missing key, as could a composing contract that writes the exported `_escrow` map directly.

Consider asserting `_encryptionKeys.member(spenderId)` before the lookup, as `approve` and `_assertOwnsEncryptionKey` already do.

N-05 Unused Proposal State in the Multisig Preset

`ShieldedMultiSigV2` imports `ProposalManager` only for its `Recipient` type. Since importing a module allocates all of its ledger fields, the import also instantiates `_nextProposalId` and `_proposals`, which nothing ever writes.

Consider moving the recipient types into a types-only module so that the preset's state layout contains only the fields it uses.

Conclusion

The audited modules add threshold-controlled custody and token issuance to the Compact Contracts library, authorized by secp256k1 ECDSA signatures over EIP-712 digests. OpenZeppelin also audited the `ZOwnableNativeShieldedToken` preset and reviewed the fixes for the previous audit.

The modules document their trust assumptions in detail, but much of a deployment's security rests on the composing contract, its operators, and standard library coin handling that the modules do not control. The audit identified one medium severity issue, in which a `ShieldedTreasury` send addressed to the treasury itself leaves the coin unspendable by the module, along with several low and note severity issues. As the multisig modules depend on Ledger v9, which was not production-ready at the time of the audit, their behavior should be confirmed against the released platform before deployment.

OpenZeppelin thanks the development team for their responsiveness and the detailed discussions throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.